



# Predicting NBA Shot Make Probability

from player-tracking data



Analyst Intern Project -- Oklahoma City Thunder

Bogdan Khudoidodov

Code: project\_code.py | Predictions: submission.csv | AI usage log: ai\_prompts.md

## 1. Objective & Summary

The goal is to predict the probability that an NBA field goal attempt is made, using player-tracking-derived features (shot location, defender positioning and closeout speed, shot clock, dribbles, etc.), trained on 425,719 shots from two seasons and scored on 213,977 held-out shots from a third season, using log-loss.

### Headline results

LightGBM reduces log-loss from a naive constant-probability baseline of 0.6896 to 0.6243 (a 9.5% relative improvement), and beats a spline logistic-regression baseline (0.6342). Calibration is excellent -- predicted probabilities track observed make rates almost exactly across the full range (see Section 6).

Approach in one paragraph: engineer basketball-motivated features from the raw tracking columns (shot geometry, defender closeout dynamics, out-of-fold target encoding of player/team/zone shooting tendencies, and interaction terms), train a LightGBM gradient-boosted-tree classifier with 5-fold cross-validation, compare against a simpler logistic-regression-with-splines baseline, and validate the whole approach with an explicit test for how well it generalizes to an unseen season -- which is exactly the situation testing.csv represents.

## 2. Data Exploration & a Critical Finding

Before modeling, I checked basic shot-making patterns against real basketball intuition, and checked how training.csv and testing.csv relate to each other.

**Sanity check:** Overall FG% 45.8%; 2PT 52.7% vs 3PT 36.1%. By shot type: dunk 88.5%, layup 53.5%, tip 48.1%, post 45.4%, floater 41.7%, jumper 37.8%, heave 9.5%. Contested shots make 43.8% vs. 62.1% uncontested.

**Season / roster overlap:** training.csv.gz has 425,719 shots across two season\_id values (fe055, 2676a); testing.csv.gz has 213,977 shots, all from a single season\_id (d7a2d) that NEVER appears in training. Team IDs fully overlap (30/30 in both), but shooter IDs only partly overlap (536 of 566 test shooters also appear in training).

### Why this matters for modeling

Because season\_id is 100% disjoint between train and test, it cannot be used as a raw feature (it would always be an unseen category at prediction time), and the originally planned 'Group K-Fold by season\_id' isn't a workable cross-validation scheme with only two training groups. I replaced it with (a) a plain Stratified K-Fold on the outcome for training/encoding, and (b) a dedicated train-on-one-season / validate-on-the-other diagnostic (Section 5) that directly measures how much the model degrades under a season shift -- the same kind of shift the real test set represents.

## 3. Feature Engineering

### 3.1 Shot geometry

Recomputed distance and angle to the rim center (-41.75, 0) from locationx/locationy. The recomputed distance matched the provided `distance` column to within  $\sim 1e-5$  ft (floating point noise only) -- a useful sanity check that units/coordinates are understood correctly. Added angle\_deg (0deg = straight on from the rim, 90deg = along the baseline) to separate a straight-on jumper from a corner-angle shot at the same raw distance, and derived a `zone` feature (restricted area / paint / mid-range / corner three / above-the-break three): restricted area = within 4 ft of the rim, paint = any other two within 14 ft, corner three = a three taken at  $\geq 62$ deg from straight-on (tight to the baseline), above-the-break three = every other three, mid-range = everything left over.

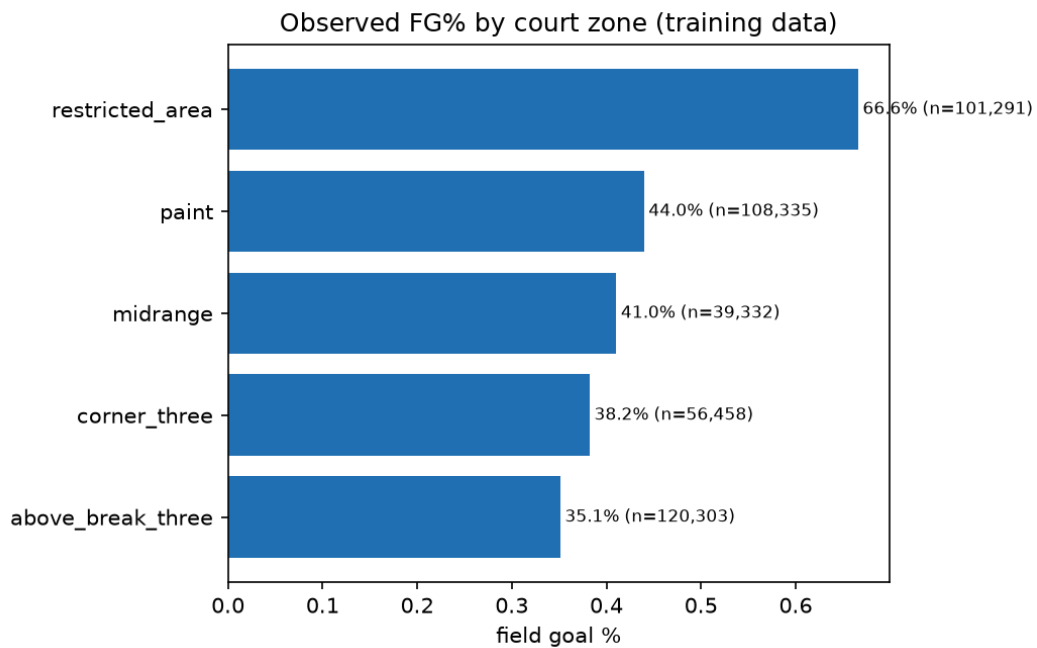


Figure 1. Observed FG% by court zone in training data -- a clean, monotonic basketball-sane ordering from the rim (66.6%) out to above-the-break threes (35.1%).

Figure 1b paints the same five numbers onto the court. The zone shapes are not hand-drawn: a fine grid of court points is run through the exact same zone function the model uses, so each colored region is precisely what that zone means in the feature (e.g. 'paint' is the 14 ft ring around the rim, not just the rectangle). Colors are centered on the league average -- red above, blue below.

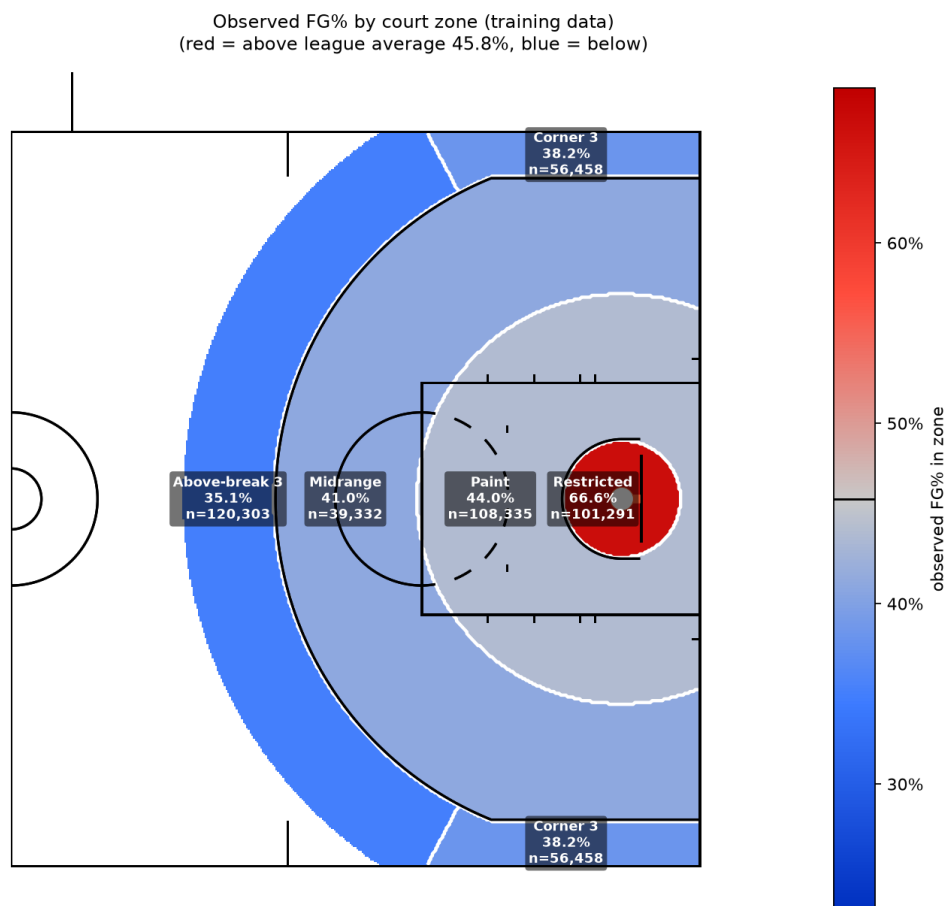


Figure 1b. The five zones of Figure 1 drawn on the court, colored by observed FG%. Only the restricted area beats the 45.8% league average; every zone further out is progressively colder, with above-the-break threes the coldest.

### 3.1.1 Hot / neutral / cold shot chart

To see this same zone effect continuously rather than in five discrete buckets, I fit a 2D Gaussian KDE separately on made and missed shot locations (the same kernel-density technique `sns.kdeplot` uses, applied per outcome class since a single KDE call can't take signed weights) and combined them into a smoothed local-FG% surface:  $\text{make\_density} / (\text{make\_density} + \text{miss\_density})$ , drawn on a `sportypy` NBA half-court -- which, conveniently, uses the exact same real-world foot coordinates as this dataset (hoop 5.25 ft from the baseline). Areas with too few nearby shots to estimate reliably are left blank rather than extrapolated.

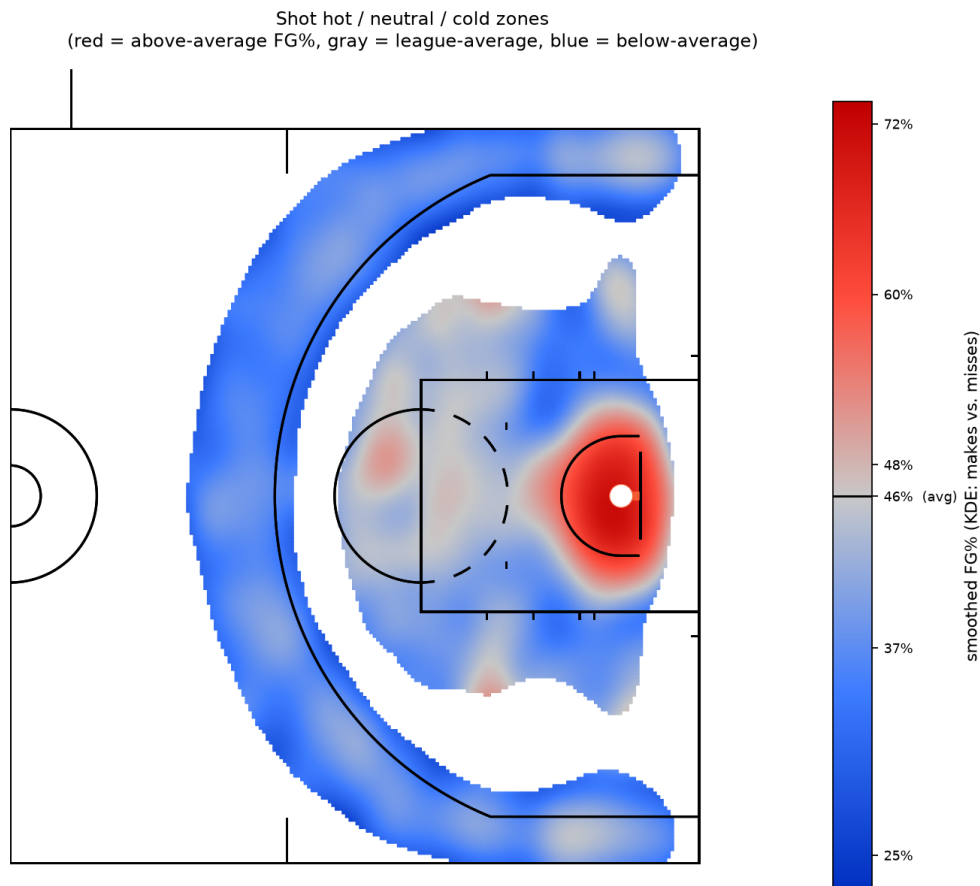


Figure 1c. Smoothed FG% by court location. Red = above the 45.8% league average, gray = at average, blue = below average. The restricted area is clearly hot; a thin cold crescent tracks the three-point line and the deep corners/wings, mirroring Figure 1's zone breakdown at every point on the floor instead of five buckets.

### 3.2 Defender closeout dynamics

`closestdefapproach` gives the closest defender's distance at 1.0s, 0.75s, 0.5s and 0.25s before the shot; `closestdefdist` gives that same distance at release. From this 5-point time series I derived: per-segment closing speeds (ft/s), overall average closing speed, closing acceleration (is the defender speeding up into the contest, or slowing down?), the minimum distance reached at any point in the sequence, and a 'recovery gap' (release distance minus the closest point reached) which distinguishes a defender who stayed attached from one who got beaten and is recovering. Only ~43% of sequences are monotonically closing -- the rest show the shooter creating separation at some point, confirming this is real defender movement, not a static distance repeated four times.

### 3.3 Contester aggregates

`num_contesters` ranges 0-4 with `distcont1..4` populated accordingly (e.g. `distcont2` is only non-null when `num_contesters >= 2`). I kept the raw `distcont1..4` columns (LightGBM handles missing values natively) and added order-independent aggregates: `contest_min_dist`, `contest_mean_dist`, `contest_max_dist`.

### 3.4 Out-of-fold target encoding (shooting tendency by shooter / defender / team / zone)

To let the model use 'this shooter makes shots at an above-average rate' or 'this zone runs cold' as a numeric feature without leaking a shot's own outcome into its own encoding, I used K-fold (K=5, stratified on outcome) leave-fold-out target encoding for shooter\_id, closestdef\_id, off\_team\_id, def\_team\_id, and zone, with additive count-based smoothing toward the global mean (so a player with 3 attempts doesn't look like a 100%-shooter). Critically, the SAME fold split is reused for both the encoding step and the model-training step, so a shot's encoded features are always derived only from the exact rows its model fold was trained on -- no leakage. For the test set, encodings are refit on 100% of training data, with unseen categories (e.g. the 30 test shooters never seen in training) falling back to the global mean.

### 3.5 Interaction terms

distance x closestdefdist, dribblesbefore x shotclock, num\_contesters x distance, avg\_closing\_speed x distance, shooterspeed x dribblesbefore, and three x closestdefdist -- basketball-motivated combinations where the marginal effect of one variable plausibly depends on another (e.g. a closing defender matters far more on a jump shot than on a shot already in flight near the rim).

## 4. Modeling & Ensembling

---

### 4.1 Why gradient-boosted trees

Shot-make probability is driven by nonlinear, interacting effects (e.g. 'far defender' helps a lot on a jumper but is almost irrelevant on a dunk already in motion). LightGBM captures this without hand-specifying every interaction, handles the natural missingness in contesters columns and categorical features (shottype, gamestate, month, zone) directly, and trains fast enough for proper cross-validated ensembling at this data size.

### 4.2 Interpretable baseline

A logistic regression with natural cubic splines on distance, closestdefdist, shotclock and angle\_deg (plus one-hot categoricals and the same engineered numeric/target-encoded features) was trained as a second, more interpretable model -- both as a sanity check on the feature engineering and as a candidate for ensembling.

### 4.3 Ensembling / blending

I searched the blend weight  $w$  in  $[0, 1]$  on  $w \cdot P_{\text{LightGBM}} + (1-w) \cdot P_{\text{LogReg}}$  against out-of-fold log-loss. The optimal weight found was  $w=1.00$  (i.e. pure LightGBM), giving a blended OOF log-loss of 0.62425. Since this did not improve on LightGBM alone, the final submission uses LightGBM only -- the logistic model is kept as a reported baseline/diagnostic rather than forced into the ensemble.

### 4.4 Hyperparameters (LightGBM)

- learning\_rate=0.03, num\_leaves=31, min\_child\_samples=150 (guards against overfitting high-cardinality encoded features), subsample=colsample\_bytree=0.8, L1/L2 regularization, up to 3000 trees with early stopping (100 rounds) on each fold's held-out split.

## 5. Validation Strategy

---

Primary validation: 5-fold Stratified K-Fold on 'outcome' (not grouped by season\_id -- see Section 2 for why). Both the target encoding and the model training/early-stopping reuse the exact same fold assignment, so OOF log-loss is a leakage-free estimate of test-like performance.

Secondary, diagnostic validation: trained on one training season and validated on the other (and vice versa), refitting target encodings on the training season only each time, to directly estimate the cost of a season shift -- the same shift the real test set represents.

| Model                                             | OOF log-loss   |
|---------------------------------------------------|----------------|
| Naive baseline (global mean)                      | 0.68963        |
| <i>constant prediction = overall training FG%</i> |                |
| Logistic Regression + splines                     | 0.63420        |
| <i>interpretable baseline</i>                     |                |
| <b>LightGBM (5-fold OOF)</b>                      | <b>0.62425</b> |
| <i>primary model</i>                              |                |
| Blend (LightGBM + LogReg)                         | 0.62425        |
| <i>best blend weight on LightGBM = 1.00</i>       |                |
| Season holdout: train fe055, val 2676a            | 0.62981        |
| Season holdout: train 2676a, val fe055            | 0.62528        |
| <b>Final submission OOF log-loss</b>              | <b>0.62425</b> |

### Reading the season-holdout gap

Average season-holdout log-loss (0.62755) is about 0.5% worse than the in-sample 5-fold CV estimate (0.62425). This is a small but real gap -- expected, since season-holdout training sees only ~213k rows (half the data) and must generalize to a wholly different set of games/rosters. I'd expect the true testing.csv score to land closer to the season-holdout number than to the plain CV number, since testing.csv is likewise a season the model never trained on.

## 6. Results & Diagnostics

### 6.1 Feature importance

distance dominates, followed by shottype and distcont1 (the first/primary contesters distance) -- unsurprising, since shot difficulty is overwhelmingly about where and what kind of shot it is. The engineered contestcount\_x\_distance interaction and the zone target encoding both rank in the top 5, validating that the feature engineering is adding real signal beyond the raw columns.

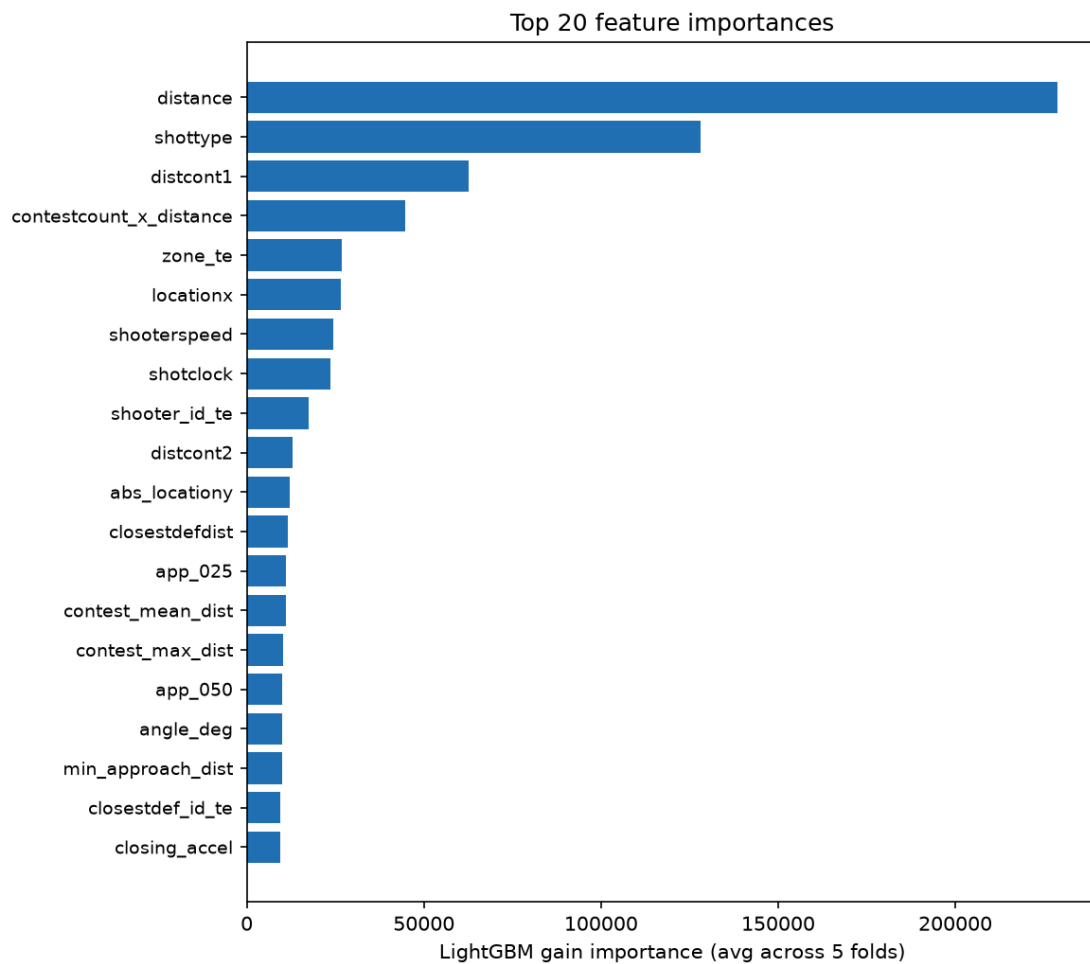


Figure 2. Top 20 features by LightGBM gain importance, averaged across the 5 CV folds.

## 6.2 SHAP-style feature impact

Using LightGBM's native TreeSHAP-equivalent contributions (pred\_contrib=True) on an 8,000-row sample, the ranking is consistent with gain importance, with shooterspeed and locationx rising in relative importance -- i.e. these features matter less often but can swing an individual prediction substantially.

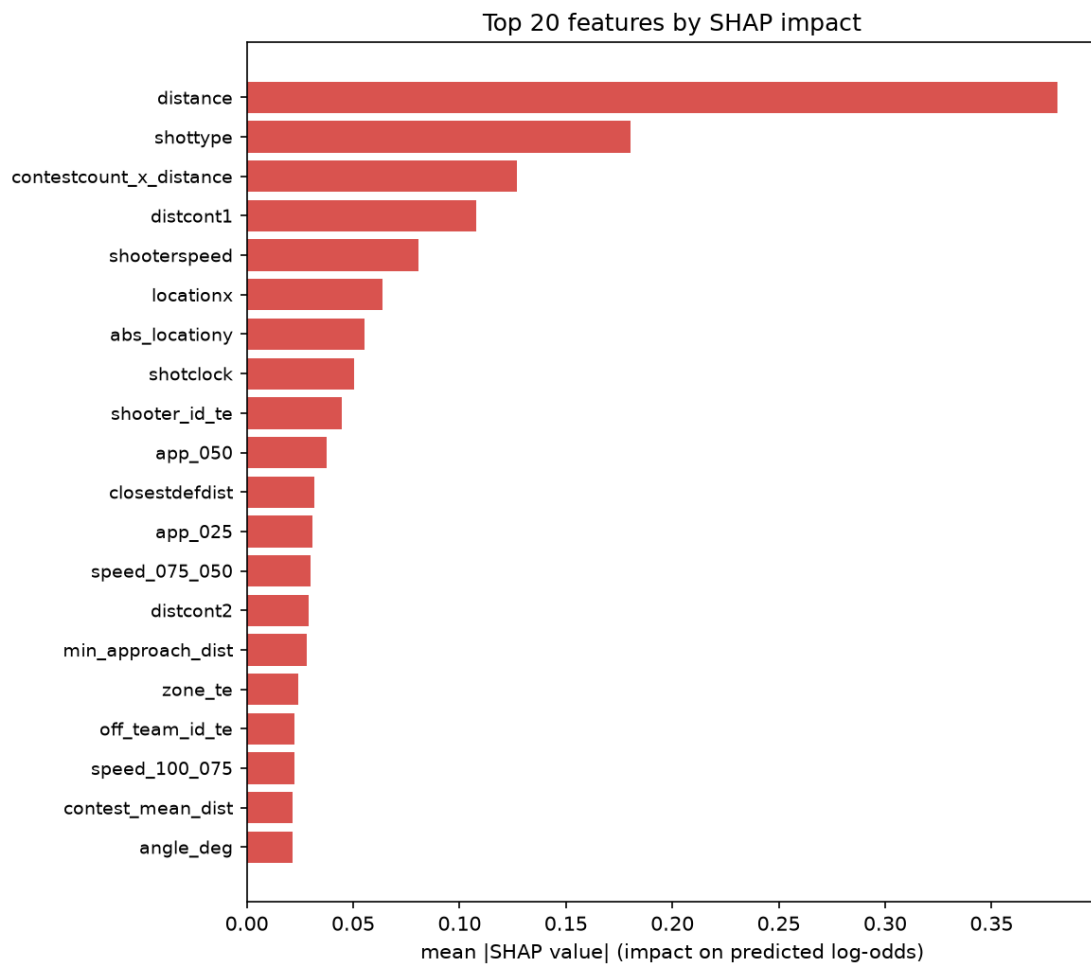


Figure 3. Top 20 features by mean |SHAP value| (impact on predicted log-odds).

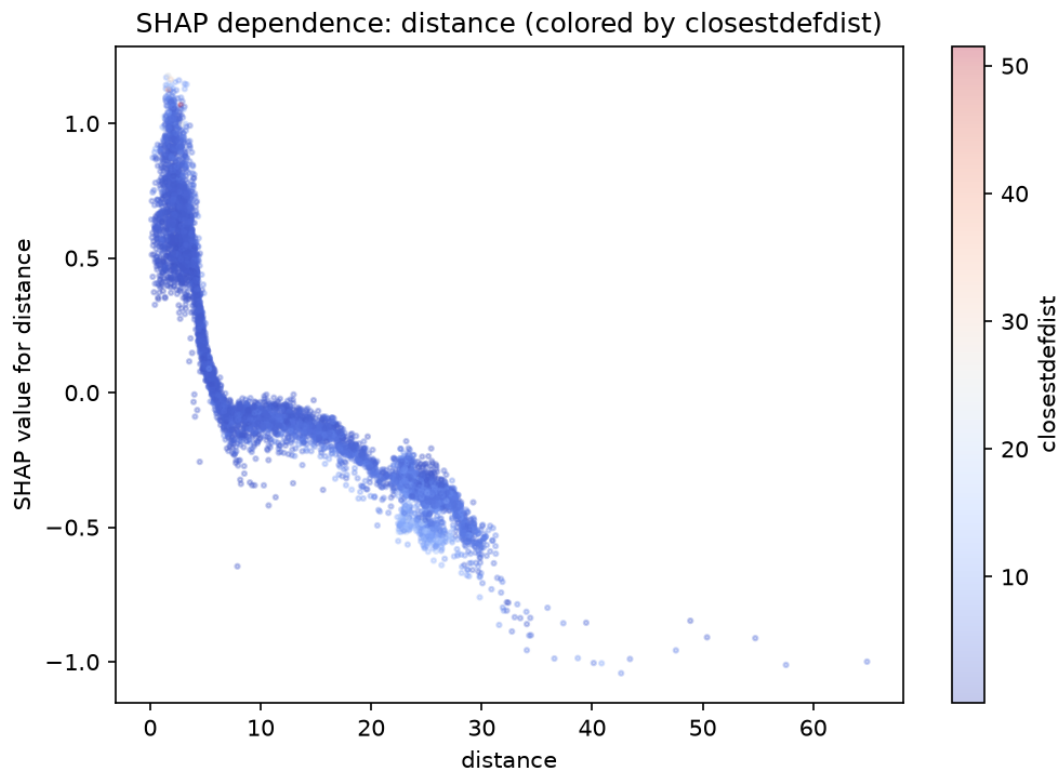


Figure 4. SHAP dependence for `distance`. The SHAP value falls smoothly with distance out to roughly 22-28 ft, where there is a visible local uptick before the curve resumes falling -- consistent with the well-known 'long two' effect: a 20-23 ft two-point jumper is genuinely a worse shot-value proposition than a three from just beyond the arc, and the model has recovered that pattern directly from the data without being told about the three-point line.

### 6.3 Calibration

Binning the out-of-fold predictions into 15 quantile groups and comparing mean predicted probability to observed make rate shows the model is very well calibrated across the full probability range -- important for a log-loss metric, which penalizes confident-but-wrong predictions heavily.

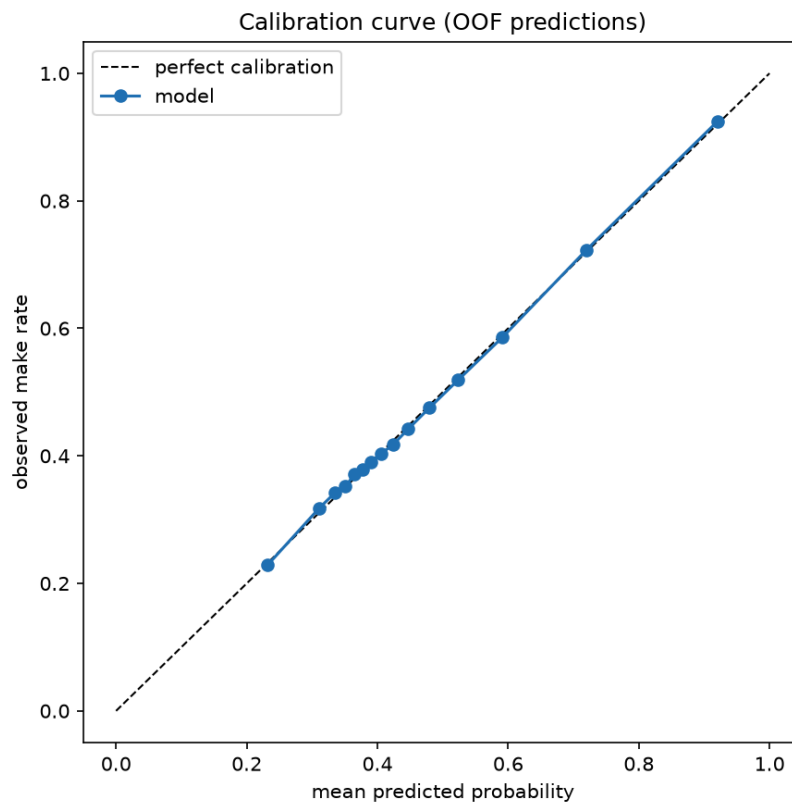


Figure 5. Calibration curve on out-of-fold predictions.

## 7. Reflections, Pitfalls & Future Work

### 7.1 Pitfalls encountered

- closestdefapproach ships as a literal string "{d1,d2,d3,d4}", not a list -- needed explicit parsing. Verified the time ordering (1.0s, 0.75s, 0.5s, 0.25s before release) by checking that values trend toward closestdefdist at release.
- The disjoint season\_id between train and test (Section 2) -- caught by directly intersecting the two sets of IDs before modeling, rather than discovering it later as an unexplained CV-vs-leaderboard gap.
- Leakage discipline for target encoding required care: the fold split used to generate out-of-fold encodings must be the exact same split used for model training, or a subtler leak reappears (a validation row's encoding would be computed from a slightly different set of rows than the model it's evaluated against was trained on).

### 7.2 What I'd try with more time or more data

- Play type / possession context (pick-and-roll vs. isolation vs. spot-up) and lineup data -- shot difficulty depends heavily on how the shot was created, which isn't directly observable here.
- Defender body orientation / closing from the shooter's blind side vs. straight on -- the tracking data gives distance but not angle-of-approach of the defender relative to the shooter's sightline.
- Screen and assist information, shooter handedness, score margin / clutch-time context, and opponent personnel (e.g. a specific rim protector on the floor).
- More seasons of history would stabilize the per-player target encodings (many shooters have modest attempt counts) and would let a proper Group K-Fold by season become feasible.
- A full Optuna/hyperparameter search and monotonic constraints on distance and closestdefdist (to guarantee the model's marginal effects never run counter to basketball logic even in sparse regions of feature space) were out of scope for this project's time budget but would be natural next steps.